

# Trabajo Práctico Especial



Proxy SOCKS5 Preentrega  
Protocolos de Comunicación 2026

## **Grupo 2**

Tiago Heras - 65627

Mateo Arias - 65613

Amador Vallejo Tapia - 65454

Manuel García Puente - 65505

**Instituto Tecnológico de Buenos Aires**

7 de julio de 2026

# Índice

|                                                                                                          |           |
|----------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas</b>                 | <b>4</b>  |
| 1.1. Arquitectura general del servidor proxy SOCKS5 . . . . .                                            | 4         |
| 1.2. Protocolo de monitoreo y configuración . . . . .                                                    | 4         |
| 1.2.1. Descripción general . . . . .                                                                     | 4         |
| 1.2.2. Decisiones de diseño . . . . .                                                                    | 5         |
| 1.2.3. Formato de los mensajes . . . . .                                                                 | 5         |
| 1.2.4. Comandos soportados . . . . .                                                                     | 5         |
| 1.2.5. Flujo de una sesión típica . . . . .                                                              | 6         |
| 1.3. Aplicación cliente de monitoreo . . . . .                                                           | 7         |
| 1.3.1. Sintaxis . . . . .                                                                                | 7         |
| 1.3.2. Ejemplos de uso . . . . .                                                                         | 7         |
| 1.4. Métricas del sistema . . . . .                                                                      | 7         |
| <b>2. Problemas encontrados durante el diseño y la implementación</b>                                    | <b>8</b>  |
| 2.1. [Aca tenemos que poner los problemas que nos surgieron] . . . . .                                   | 8         |
| <b>3. Limitaciones de la aplicación</b>                                                                  | <b>8</b>  |
| 3.1. Registro de accesos bloqueante . . . . .                                                            | 8         |
| 3.2. Autenticación del protocolo de monitoreo . . . . .                                                  | 8         |
| 3.3. Backlog de conexiones . . . . .                                                                     | 8         |
| 3.4. Soporte limitado de comandos SOCKS . . . . .                                                        | 9         |
| 3.5. [Aca podemos agregar mas limitaciones y quizas podemos arreglar algunas de las de arriba] . . . . . | 9         |
| <b>4. Posibles extensiones</b>                                                                           | <b>9</b>  |
| <b>5. Conclusiones</b>                                                                                   | <b>9</b>  |
| <b>6. Ejemplos de prueba</b>                                                                             | <b>9</b>  |
| 6.1. Prueba básica del proxy SOCKS5 . . . . .                                                            | 9         |
| 6.2. Prueba de monitoreo . . . . .                                                                       | 10        |
| 6.3. Prueba de graceful shutdown . . . . .                                                               | 10        |
| <b>7. Guía de instalación</b>                                                                            | <b>10</b> |
| 7.1. Requisitos del sistema . . . . .                                                                    | 10        |
| 7.2. Compilación . . . . .                                                                               | 11        |
| 7.3. Limpieza . . . . .                                                                                  | 11        |
| <b>8. Instrucciones para la configuración</b>                                                            | <b>11</b> |
| 8.1. Opciones del servidor . . . . .                                                                     | 11        |
| 8.2. Ejemplos de configuración . . . . .                                                                 | 12        |
| 8.3. Opciones del cliente de monitoreo . . . . .                                                         | 12        |
| <b>9. Ejemplos de configuración y monitoreo</b>                                                          | <b>12</b> |
| 9.1. Escenario 1: Servidor básico con monitoreo local . . . . .                                          | 12        |
| 9.2. Escenario 2: Acceso a un sitio bloqueado . . . . .                                                  | 13        |

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>10.Documento de diseño del proyecto</b>                              | <b>13</b> |
| 10.1. Diagrama de módulos . . . . .                                     | 13        |
| 10.2. Descripción de los componentes . . . . .                          | 14        |
| 10.2.1. Selector de eventos ( <code>selector.c/h</code> ) . . . . .     | 14        |
| 10.2.2. Buffer circular ( <code>buffer.c/h</code> ) . . . . .           | 15        |
| 10.2.3. Máquina de estados ( <code>stm.c/h</code> ) . . . . .           | 15        |
| 10.2.4. Parsers SOCKS5 . . . . .                                        | 15        |
| 10.2.5. Servidor de monitoreo ( <code>mng_server.c/h</code> ) . . . . . | 15        |
| 10.2.6. Cliente de monitoreo ( <code>client.c</code> ) . . . . .        | 16        |
| 10.3. Diagrama de flujo de una conexión SOCKS5 . . . . .                | 16        |

# 1 Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas

## 1.1 Arquitectura general del servidor proxy SOCKS5

El servidor proxy SOCKS5 está implementado como una aplicación monohilo que utiliza multiplexación de entrada/salida no bloqueante mediante un selector de eventos (implementado en `selector.c`). Todas las conexiones entrantes y salientes se manejan en un único hilo de ejecución, con la única excepción de la resolución de nombres de dominio (FQDN), que se delega a un hilo auxiliar mediante `getaddrinfo(3)`.

Cada conexión SOCKS5 se modela como una máquina de estados (implementada en `stm.c`) que transiciona a través de las siguientes fases:

```
HELLO_READ -> HELLO_WRITE -> [AUTH_READ -> AUTH_WRITE] -> REQUEST_READ
      -> REQUEST_RESOLV -> REQUEST_CONNECTING -> REQUEST_WRITE -> COPY
      -> DONE / ERROR
```

Descripción de cada fase:

- **HELLO**: El cliente envía su versión SOCKS y los métodos de autenticación que soporta. El servidor selecciona el método según su política interna (sin autenticación si no hay usuarios registrados, usuario/contraseña en caso contrario).
- **AUTH (RFC 1929)**: Si se seleccionó el método de usuario/contraseña, se intercambian las credenciales y se validan contra la tabla de usuarios del servidor.
- **REQUEST**: El cliente envía el pedido de conexión (comando CONNECT, tipo de dirección y destino). El servidor parsea la solicitud, resuelve el FQDN si es necesario (en un hilo separado), y establece una conexión TCP no bloqueante hacia el servidor de origen.
- **COPY**: Una vez establecida la conexión, se inicia un túnel bidireccional que copia datos entre el cliente y el servidor de origen hasta que alguna de las partes cierra la conexión.

## 1.2 Protocolo de monitoreo y configuración

### 1.2.1. Descripción general

El protocolo de monitoreo es un protocolo de texto simple sobre TCP, diseñado para permitir la administración remota del servidor proxy en tiempo de ejecución sin necesidad de reiniciar el servicio. Escucha en un socket pasivo independiente del proxy SOCKS5, en un puerto configurable (por defecto 8080).

### 1.2.2. Decisiones de diseño

Se optó por un protocolo basado en texto plano (en lugar de binario) por las siguientes razones:

- **Simplicidad de implementación:** Los comandos son legibles por humanos, lo que facilita la depuración y las pruebas manuales.
- **Interoperabilidad:** Puede probarse con herramientas estándar como `nc` (netcat) o `telnet`.
- **Baja complejidad:** Dado que el protocolo maneja pocos comandos y respuestas cortas, la sobrecarga del formato texto es despreciable.
- **Extensibilidad:** Agregar nuevos comandos es trivial.

### 1.2.3. Formato de los mensajes

**Formato de los comandos (cliente → servidor)** Cada comando es una línea de texto ASCII terminada en el carácter `0x0A` (`\n`). Opcionalmente puede incluir el carácter `0x0D` (`\r`) antes del salto de línea, el cual es ignorado por el servidor.

```

1 comando      = verbo *(SP argumento) CRLF
2 verbo        = 1*ALPHA
3 argumento     = 1*(%x21-7E)    ; caracteres imprimibles no espacio
4 CRLF         = [CR] LF
5 CR           = %x0D
6 LF           = %x0A
7 SP           = %x20
```

Listing 1: Gramática del protocolo de monitoreo (formato ABNF)

**Formato de las respuestas (servidor → cliente)** Cada respuesta es una línea de texto ASCII terminada en CRLF. El primer token indica éxito o fracaso:

- `+OK` seguido de datos opcionales: el comando se ejecutó correctamente.
- `-ERR` seguido de un mensaje de error: el comando falló.

```

1 respuesta     = status [SP datos] CRLF
2 status        = "+OK" / "-ERR"
3 datos         = 1*(%x20-7E)
```

Listing 2: Gramática de las respuestas

### 1.2.4. Comandos soportados

#### 1. GET\_METRICS

- **Propósito:** Obtiene las métricas actuales del servidor.
- **Sintaxis:** `GET_METRICS\n`

- **Respuesta exitosa:** +OK ACT:<n>HIST:<n>BYTES:<n>\r\n
- **Campos:**
  - ACT: Cantidad de conexiones activas en este momento.
  - HIST: Cantidad histórica de conexiones desde el inicio del servidor.
  - BYTES: Cantidad total de bytes transferidos.

## 2. ADD\_USER

- **Propósito:** Agrega un nuevo usuario autorizado para usar el proxy.
- **Sintaxis:** ADD\_USER <usuario><contraseña>\n
- **Respuesta exitosa:** +OK\r\n
- **Respuesta de error:** -ERR <motivo>\r\n
- **Restricciones:** Máximo 10 usuarios. El nombre de usuario y la contraseña no pueden superar los 255 caracteres cada uno.

## 3. DEL\_USER

- **Propósito:** Elimina un usuario previamente registrado.
- **Sintaxis:** DEL\_USER <usuario>\n
- **Respuesta exitosa:** +OK\r\n
- **Respuesta de error:** -ERR user not found\r\n

### 1.2.5. Flujo de una sesión típica

|    | Cliente:                   | Servidor:                |
|----|----------------------------|--------------------------|
| 1  | CONECTAR (TCP)             |                          |
| 2  |                            |                          |
| 3  | ----->                     |                          |
| 4  | GET_METRICS\n              |                          |
| 5  | ----->                     |                          |
| 6  |                            | +OK ACT:3 HIST:150 BYTES |
|    | :1048576\r\n               |                          |
| 7  | <-----                     |                          |
| 8  | ADD_USER alberto pass123\n |                          |
| 9  | ----->                     |                          |
| 10 |                            | +OK\r\n                  |
| 11 | <-----                     |                          |
| 12 | DEL_USER alberto\n         |                          |
| 13 | ----->                     |                          |
| 14 |                            | +OK\r\n                  |
| 15 | <-----                     |                          |
| 16 | CERRAR CONEXIÓN            |                          |
| 17 | ----->                     |                          |

Listing 3: Ejemplo de sesión de monitoreo

## 1.3 Aplicación cliente de monitoreo

El cliente de monitoreo es una aplicación de línea de comandos que utiliza I/O bloqueante (dada la simplicidad del protocolo). Se conecta al servidor de management, envía el comando solicitado y muestra la respuesta.

### 1.3.1. Sintaxis

```
1 ./bin/client [opciones] <comando>
2
3 Opciones:
4 -L <dirección>   Dirección IP del servidor de management (defecto
                    : 127.0.0.1)
5 -P <puerto>     Puerto del servidor de management (defecto:
                    8080)
6
7 Comandos:
8 get-metrics      Obtiene métricas del servidor
9 add-user <usr> <pass>   Agrega un usuario
10 del-user <usr>      Elimina un usuario
```

Listing 4: Uso del cliente de monitoreo

### 1.3.2. Ejemplos de uso

```
1 $ ./bin/client get-metrics
2 +OK ACT:0 HIST:12 BYTES:409600
3
4 $ ./bin/client add-user juan pass123
5 +OK
6
7 $ ./bin/client del-user juan
8 +OK
```

Listing 5: Ejemplos del cliente de monitoreo

## 1.4 Métricas del sistema

El servidor recolecta las siguientes métricas, accesibles a través del comando `GET_METRICS`:

- **Conexiones activas (ACT)**: Número de conexiones SOCKS5 actualmente en curso. Se incrementa al aceptar una nueva conexión y se decrementa al finalizar.
- **Conexiones históricas (HIST)**: Contador acumulativo de todas las conexiones SOCKS5 establecidas desde el inicio del servidor.
- **Bytes transferidos (BYTES)**: Suma total de bytes transferidos en ambas direcciones (cliente → proxy y proxy → origen) a través del túnel.

Todas las métricas son volátiles: se pierden al reiniciar el servidor.

## 2 Problemas encontrados durante el diseño y la implementación

### 2.1 [Aca tenemos que poner los problemas que nos surgieron]

- **Manejo de lecturas/escrituras parciales:** Durante la fase de COPY, fue necesario implementar un mecanismo robusto que maneje correctamente las lecturas y escrituras parciales, propagando los cierres de conexión de forma adecuada entre ambas mitades del túnel.
- **Resolución de nombres no bloqueante:** Dado que `getaddrinfo(3)` es una operación bloqueante, se optó por ejecutarla en un hilo separado y notificar al selector mediante `selector_notify_block`. Esto requirió cuidado para evitar condiciones de carrera en el acceso a las estructuras compartidas.
- [Terminar con cosas que tuvimos problemas]

## 3 Limitaciones de la aplicación

### 3.1 Registro de accesos bloqueante

El registro de accesos (`log_access`) escribe en el archivo `access.log` utilizando operaciones de E/S bloqueantes (`fopen`, `fprintf`, `fclose`) dentro del bucle principal de eventos no bloqueante. Esto puede introducir micro-pausas que degradan el rendimiento bajo carga elevada. Una mejora posible sería implementar un buffer de logs en memoria con escritura asíncrona.

### 3.2 Autenticación del protocolo de monitoreo

El servidor de management no implementa autenticación. Cualquier entidad que pueda conectarse al puerto de monitoreo (por defecto 8080 en 127.0.0.1) puede agregar o eliminar usuarios del proxy. Si bien el puerto de monitoreo está configurado por defecto para escuchar solo en la interfaz loopback, esta sigue siendo una limitación de seguridad.

### 3.3 Backlog de conexiones

El socket pasivo del proxy SOCKS5 se configura con un backlog de 20 conexiones (`listen(server, 20)`). Para soportar 500 clientes concurrentes de forma robusta, sería recomendable utilizar `SOMAXCONN` (generalmente 128 o superior).



### 3.4 Soporte limitado de comandos SOCKS

Actualmente solo se implementa el comando CONNECT. Los comandos BIND y UDP ASSOCIATE definidos en RFC 1928 no están soportados.

### 3.5 [Aca podemos agregar mas limitaciones y quizas podemos arreglar algunas de las de arriba]

## 4 Posibles extensiones

- **Dissector de credenciales (ettercap-like):** Implementar un módulo de inspección de tráfico en la fase COPY que detecte credenciales enviadas en texto plano en protocolos como POP3, FTP, HTTP Basic, SMTP, etc. Esta funcionalidad está prevista para la segunda entrega del trabajo práctico.
- **Soporte BIND y UDP ASSOCIATE:** Completar la implementación de los comandos faltantes del RFC 1928.
- **Autenticación del protocolo de monitoreo:** Agregar un mecanismo de autenticación (contraseña compartida, token, o autenticación mutua) para el servidor de management.
- **Logging asincrónico:** Reemplazar el registro de accesos bloqueante por un esquema con buffer circular y escritura diferida.
- **Configuración dinámica extendida:** Permitir cambiar en tiempo de ejecución otros parámetros como timeouts, tamaño de buffers, límite de conexiones concurrentes, etc.
- **Estadísticas avanzadas:** Agregar métricas como tiempo de vida promedio de conexiones, throughput por usuario, histogramas de tamaño de transferencias, etc.

## 5 Conclusiones

[Aca pongamos unas conclusiones entre todos si quieren]

## 6 Ejemplos de prueba

### 6.1 Prueba básica del proxy SOCKS5

```
1 # Iniciar el servidor con un usuario
2 $ ./bin/server -u testuser:testpass
3 Listening on TCP port 1080
4 Management listening on TCP port 8080
```

Listing 6: Iniciar el servidor proxy

```
1 # Solicitud HTTP a través del proxy SOCKS5 con autenticación
2 $ curl --socks5 127.0.0.1:1080 --proxy-user testuser:testpass \
3     http://example.com
4 <!doctype html>
5 <html>
6 ...
```

Listing 7: Probar el proxy con curl

```
1 # Iniciar sin usuarios registrados (acepta "sin autenticación")
2 $ ./bin/server
3 $ curl --socks5 127.0.0.1:1080 http://example.com
```

Listing 8: Probar el proxy sin autenticación

## 6.2 Prueba de monitoreo

```
1 $ ./bin/client get-metrics
2 +OK ACT:1 HIST:5 BYTES:24576
```

Listing 9: Consultar métricas

```
1 $ ./bin/client add-user nuevo pass123
2 +OK
3 $ ./bin/client del-user nuevo
4 +OK
```

Listing 10: Agregar y eliminar usuarios en caliente

## 6.3 Prueba de graceful shutdown

```
1 $ kill -TERM <PID_del_servidor>
2 # El servidor imprime:
3 # graceful shutdown: no se aceptan nuevas conexiones, esperando N
   conexiones activas
4 # (espera a que terminen todas las conexiones activas y luego
   finaliza)
```

Listing 11: Enviar señal SIGTERM al servidor

# 7 Guía de instalación

## 7.1 Requisitos del sistema

- Sistema operativo: Linux (kernel 4.x o superior)
- Compilador: GCC 4.9 o superior (compatible con C11)

- GNU Make 3.81 o superior
- Bibliotecas: libc estándar, pthreads

## 7.2 Compilación

```
1 $ make clean && make all
```

Listing 12: Compilar el proyecto desde la raíz

Este comando genera los siguientes artefactos:

- `bin/server` — Servidor proxy SOCKS5
- `bin/client` — Cliente de monitoreo

Los archivos objeto intermedios se almacenan en el directorio `obj/`.

## 7.3 Limpieza

```
1 $ make clean
```

Listing 13: Eliminar artefactos compilados

# 8 Instrucciones para la configuración

## 8.1 Opciones del servidor

```
1 ./bin/server [OPCIONES]
2
3 Opciones:
4 -h                Imprime la ayuda y termina.
5 -l <dirección>    Dirección donde servirá el proxy SOCKS (defecto:
6                   0.0.0.0).
7 -L <dirección>    Dirección donde servirá el servicio de
8                   management
9                   (defecto: 127.0.0.1).
10 -p <puerto>      Puerto entrante para conexiones SOCKS (defecto:
11                   1080).
12 -P <puerto>      Puerto entrante para conexiones de management
13                   (defecto: 8080).
14 -u <usr>:<pass>    Usuario y contraseña para el proxy. Hasta 10
15                   ocurrencias.
16 -v                Imprime la versión y termina.
```

Listing 14: Uso del servidor

## 8.2 Ejemplos de configuración

```
1 # Servidor SOCKS en puerto 2080, management en puerto 9090
2 $ ./bin/server -p 2080 -P 9090
3
4 # Servidor con dos usuarios predefinidos
5 $ ./bin/server -u alice:pass1 -u bob:pass2
6
7 # Servidor con management en una interfaz específica
8 $ ./bin/server -L 192.168.1.100 -P 9090
```

Listing 15: Servidor con configuración personalizada

## 8.3 Opciones del cliente de monitoreo

```
1 ./bin/client [OPCIONES] <comando>
2
3 Opciones:
4   -L <dirección>   Dirección IP del servidor de management (defecto
5                     : 127.0.0.1)
6   -P <puerto>      Puerto del servidor de management (defecto:
7                     8080)
8
9 Comandos:
10  get-metrics        Obtiene métricas del servidor
11  add-user <usr> <pass>  Agrega un usuario
12  del-user <usr>       Elimina un usuario
```

Listing 16: Uso del cliente

# 9 Ejemplos de configuración y monitoreo

## 9.1 Escenario 1: Servidor básico con monitoreo local

```
1 # Terminal 1: Iniciar el servidor
2 $ ./bin/server -u admin:admin123
3
4 # Terminal 2: Consultar métricas iniciales
5 $ ./bin/client get-metrics
6 +OK ACT:0 HIST:0 BYTES:0
7
8 # Terminal 2: Agregar un usuario nuevo
9 $ ./bin/client add-user desarrollador pass123
10 +OK
11
12 # Terminal 2: Verificar que el usuario se agregó (conectándose al
13   proxy)
14 $ curl --socks5 127.0.0.1:1080 --proxy-user desarrollador:pass123 \
```

```

14      https://httpbin.org/ip
15
16 # Terminal 2: Consultar métricas después de la conexión
17 $ ./bin/client get-metrics
18 +OK ACT:0 HIST:1 BYTES:2048

```

Listing 17: Inicialización y monitoreo

## 9.2 Escenario 2: Acceso a un sitio bloqueado

```

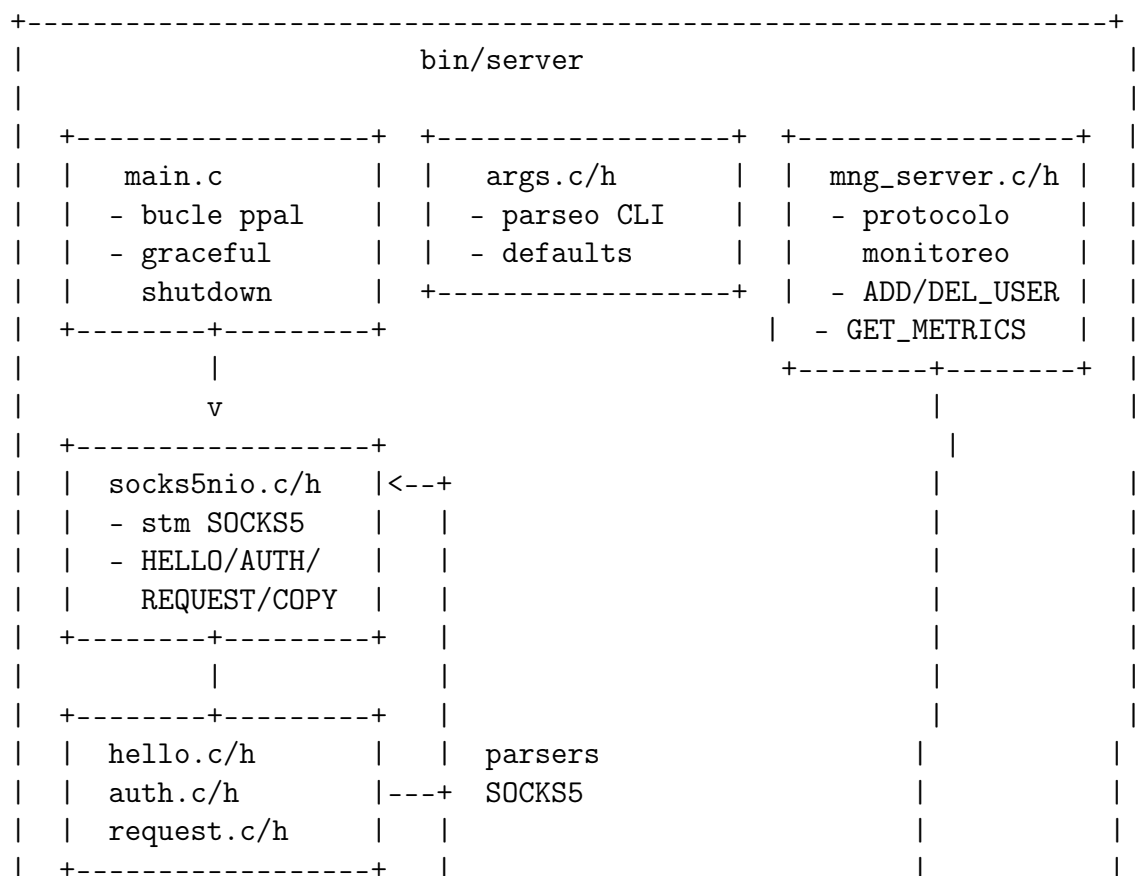
1 # Después de varias conexiones, revisar el archivo de log
2 $ cat access.log
3 [2026-07-06 20:15:30] User: admin | FQDN: example.com | IP Destino:
   93.184.216.34:80
4 [2026-07-06 20:16:45] User: admin | FQDN: httpbin.org | IP Destino:
   3.220.72.84:443

```

Listing 18: Verificación del registro de accesos

# 10 Documento de diseño del proyecto

## 10.1 Diagrama de módulos





## 10.2 Descripción de los componentes

### 10.2.1. Selector de eventos (selector.c/h)

El selector es el núcleo del multiplexado de E/S. Permite registrar descriptores de archivo con intereses de lectura/escritura y notifica mediante callbacks cuando los eventos ocurren. Soporta notificación de operaciones bloqueantes (como resolución de DNS) mediante señales.

#### Interfaz pública:

- `selector_init()`: Inicializa la librería con una señal para notificaciones internas.
- `selector_new(cantidad)`: Crea una nueva instancia de selector con capacidad para cantidad descriptores.
- `selector_register()`: Registra un fd con un handler y un interés inicial.
- `selector_select()`: Espera eventos y dispara los callbacks correspondientes.
- `selector_notify_block()`: Notifica al selector que una operación bloqueante (ej: DNS) finalizó.

### 10.2.2. Buffer circular (`buffer.c/h`)

Buffer con acceso directo a memoria que mantiene punteros de lectura y escritura separados. Permite operaciones de I/O eficientes al exponer directamente las direcciones de memoria donde leer o escribir, evitando copias intermedias.

#### Características:

- Compactación automática cuando los punteros se igualan.
- Sin asignaciones dinámicas de memoria (usa buffers pre-asignados).
- Ideal para operaciones de `recv(2)` y `send(2)` no bloqueantes.

### 10.2.3. Máquina de estados (`stm.c/h`)

Motor de máquina de estados genérico donde los eventos son los del selector (`on_read_ready`, `on_write_ready`, `on_block_ready`). Cada estado define callbacks de llegada, salida y manejo de eventos.

#### Estados definidos para SOCKS5:

- `HELLO_READ`, `HELLO_WRITE`
- `AUTH_READ`, `AUTH_WRITE`
- `REQUEST_READ`, `REQUEST_RESOLV`, `REQUEST_CONNECTING`, `REQUEST_WRITE`
- `COPY`
- `DONE`, `ERROR`

### 10.2.4. Parsers SOCKS5

- **hello.c/h**: Parser del mensaje HELLO (RFC 1928, sección 3). Procesa versión, cantidad de métodos y lista de métodos de autenticación. Incluye callback por método para selección dinámica.
- **auth.c/h**: Parser de autenticación usuario/contraseña (RFC 1929). Procesa los campos `VER`, `ULEN`, `UNAME`, `PLEN`, `PASSWD`.
- **request.c/h**: Parser del mensaje REQUEST (RFC 1928, sección 4). Soporta direcciones IPv4, IPv6 y FQDN. Incluye serialización de respuestas con `BND.ADDR`.

### 10.2.5. Servidor de monitoreo (`mng_server.c/h`)

Implementa el protocolo de monitoreo como una máquina de estados simple (`MNG_READ` → `MNG_EXECUTE` → `MNG_WRITE` → `MNG_DONE`). Utiliza el mismo selector de eventos que el proxy SOCKS5, integrándose sin necesidad de hilos adicionales.

### 10.2.6. Cliente de monitoreo (client.c)

Aplicación de línea de comandos con I/O bloqueante. Conecta por TCP al servidor de management, envía el comando y muestra la respuesta. Soporta los comandos `get-metrics`, `add-user` y `del-user`.

## 10.3 Diagrama de flujo de una conexión SOCKS5

